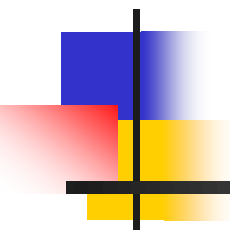
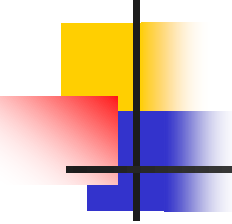


Database Systems and Data-Driven Applications



Components of a DBMS,
Disks and Files



This Lecture...

- Components of a DBMS...
- Data Storage: Disks and Files...

Structure of a DBMS

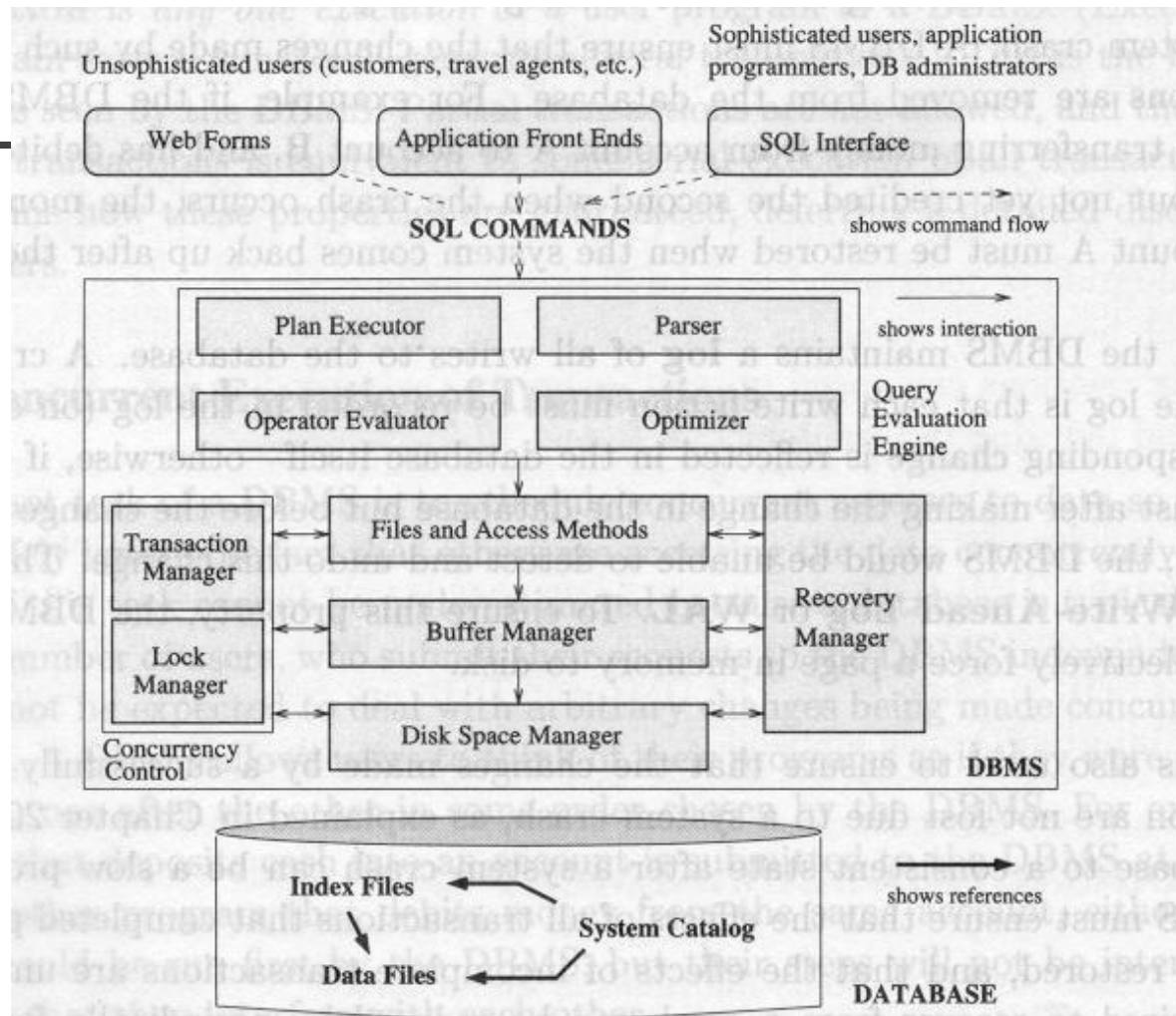
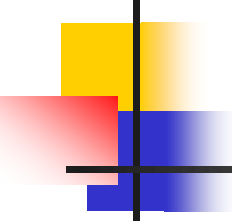
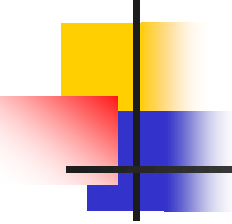


Figure 1.3 Architecture of a DBMS



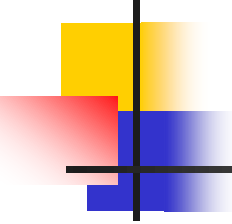
Structure of a DBMS... (contd.)

- DBMSs accept SQL statements
- The queries are **parsed** and presented to the **optimizer** (for query optimization)
- An **execution plan** is produced after optimization
- **Execution plan** consists of a set of **relational operators** and extra information



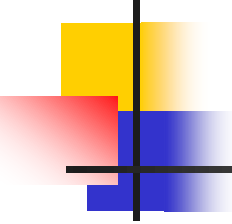
Structure of a DBMS... (contd.)

- All data is stored in logical storage called **files**
- **Files and Access Methods** provide an interface to manipulate files
- **Files** are considered to be a set of **pages**
- **Buffer manager** brings pages from disk to main memory



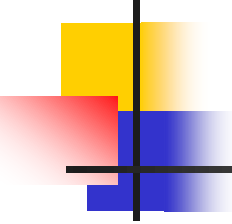
Structure of a DBMS... (contd.)

- **Disk Space Manger** deals with the management of space on disk.
- DBMS supports concurrency control and crash recovery
- This requires users scheduling the transactions carefully and maintaining a log of all changes to the database



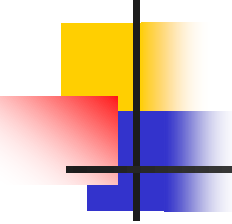
Structure of a DBMS... (contd.)

- DBMS components for transactions processing include
 - **Transaction Manager:** ensures that transactions requests/releases locks according to a appropriate locking protocol
 - **Lock Manager:** Keeps track of locks for each database object
 - **Recovery Manager:** Maintains the log and restores the system after a crash



Data Storage: Disks and Files

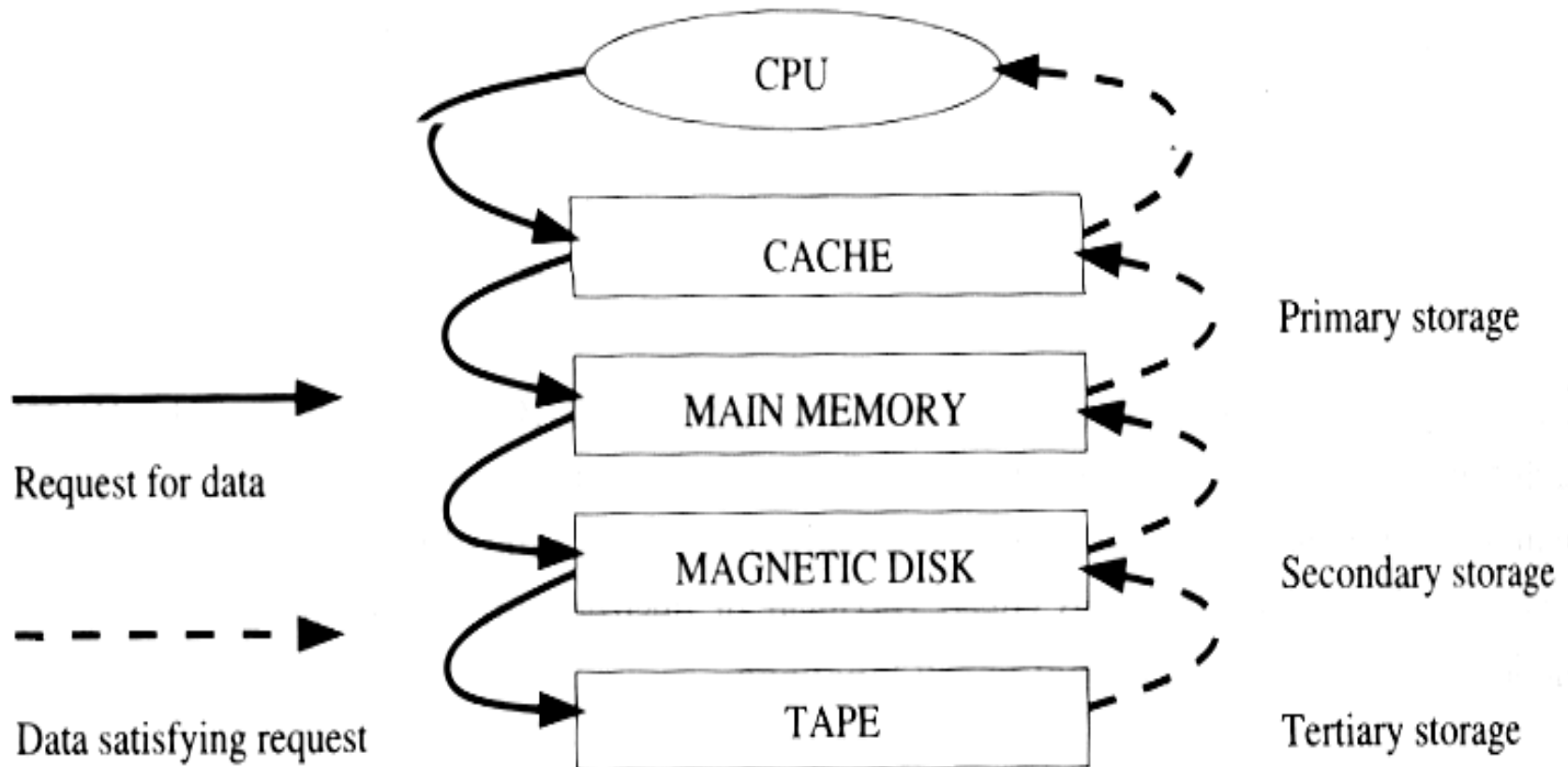
- DBMS stores information on (“hard”) disks.
- This has major implications for DBMS design!
 - **READ:** transfer data from disk to main memory (RAM).
 - **WRITE:** transfer data from RAM to disk.
 - Both are high-cost operations, relative to in-memory operations, so must be planned carefully!



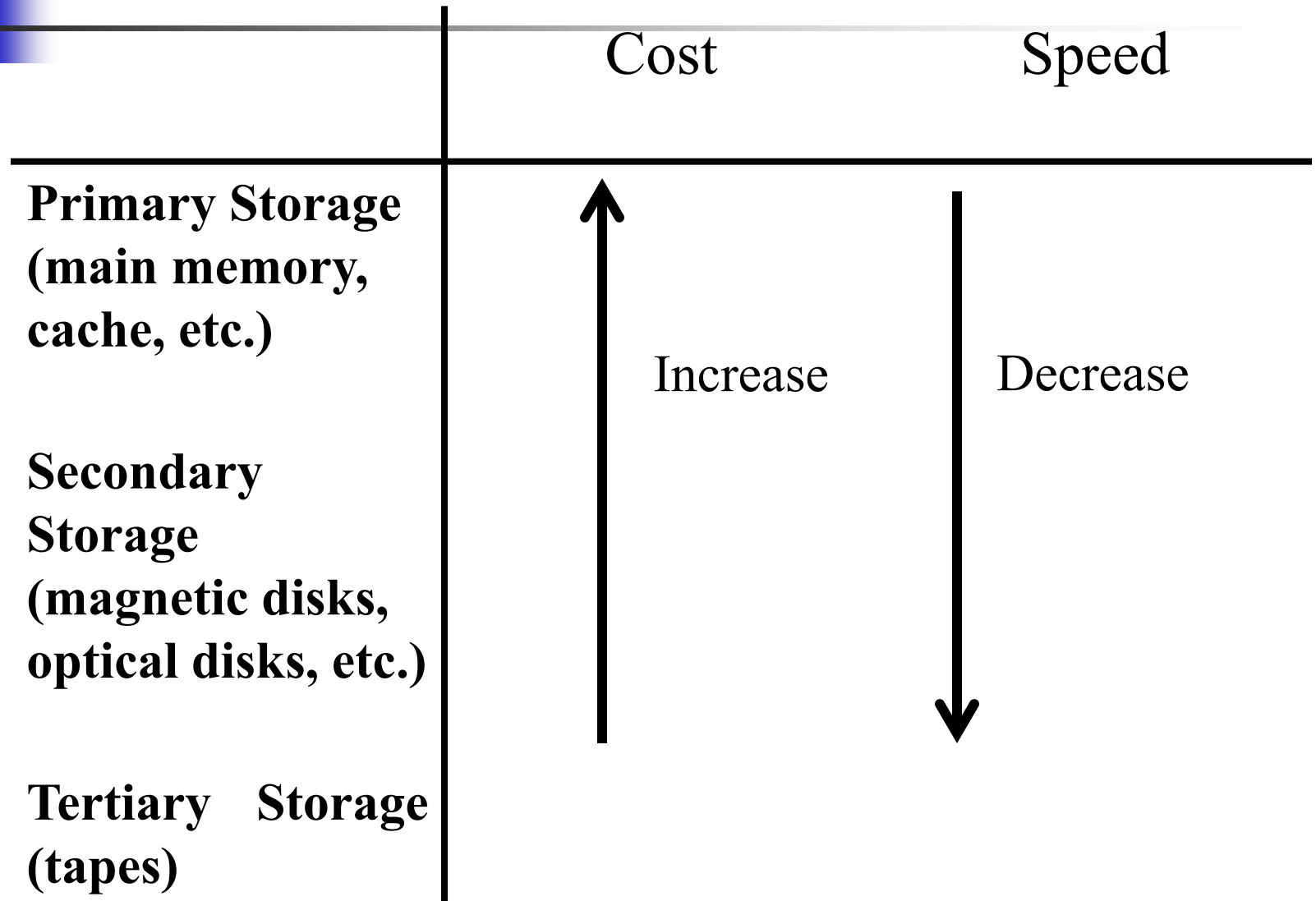
Why Not Store Everything in Main Memory?

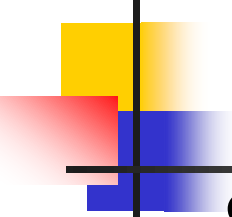
- *Costs too much.* Rs. 9000 will buy you either 256MB of RAM or 40GB of disk.
- *Main memory is volatile.* We want data to be saved between runs. (Obviously!)
- Typical storage hierarchy:
 - Main memory (RAM) for currently used data (primary storage).
 - Disk for the main database (secondary storage).
 - Tapes for archiving older versions of the data (tertiary storage).

Memory Hierarchy



Characteristics of Storage Medium





Disks

- Secondary storage device of choice.
- Main advantage over tapes: *random access* vs. *sequential*.
- Data is stored and retrieved in units called *disk blocks* or *pages*.
- Unlike RAM, time to retrieve a disk page varies depending upon location on disk.
 - Therefore, relative placement of pages on disk has major impact on DBMS performance!

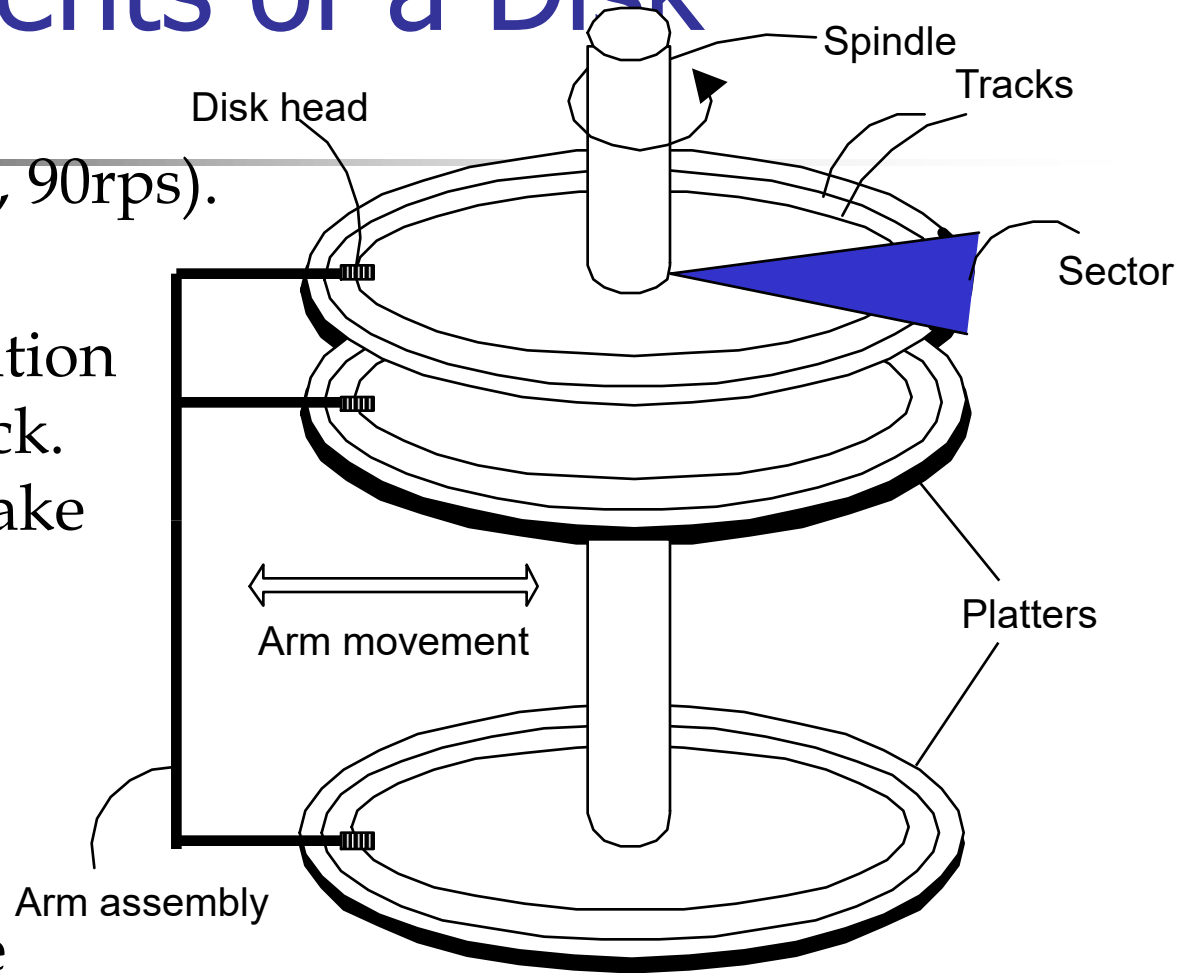
Components of a Disk

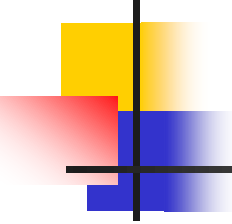
- The platters spin (say, 90rps).

- The arm assembly is moved in or out to position a head on a desired track. Tracks under heads make a *cylinder* (imaginary!).

- Only one head reads/writes at any one time.

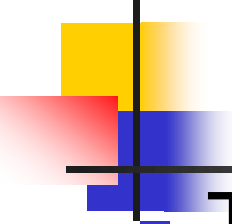
- *Block size* is a multiple of *sector size* (which is fixed).





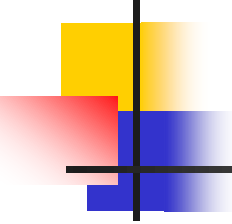
Components of a Disk

- A **disk controller** is an interface which controls the disk drive.
- This interface commands to read/write sectors by moving the arm assembly and transferring data to/from the disk surfaces
- A well known interface for PCs is SCSI (Small Computer Storage Interface)



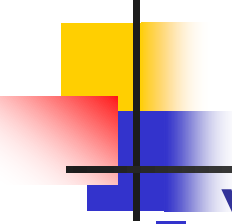
Accessing a Disk Page

- Time to access (read/write) a disk block:
 - *seek time* (moving arms to position disk head on track)
 - *rotational delay* (waiting for block to rotate under head)
 - *transfer time* (actually moving data to/from disk surface)



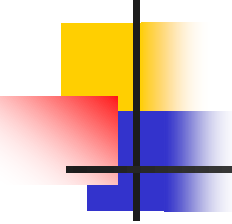
Disk Performance

- Seek time and rotational delay dominate.
 - Seek time varies from about 1 to 20msec
 - Rotational delay varies from 0 to 10msec
 - Transfer rate is about 1msec per 4KB page
- Key to lower I/O cost: **reduce seek/rotation delays!** Hardware vs. software solutions?



Disk Performance... (contd.)

- *'Next'* block concept:
 - blocks on same track, followed by
 - blocks on same cylinder, followed by
 - blocks on adjacent cylinder
- Blocks in a file should be arranged sequentially on disk (by *'next'*), to minimize seek and rotational delay.
- For a *sequential scan*, *pre-fetching* several pages at a time is a big win!



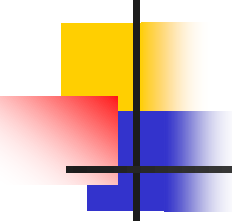
RAID

- Disks are a potential bottleneck for performance

Rate of Performance Increase

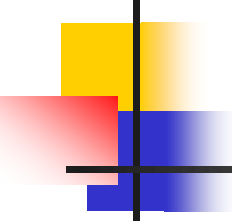
- Disk access time $\sim 10\%$ per year
- Microprocessor $> 50\%$ per year

Also, disks are not very reliable...



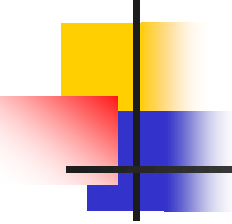
RAID... (contd.)

- Disk Array: Arrangement of several disks that gives abstraction of a single, large disk.
- Goals: Increase performance and reliability.
- Two main techniques:
 - Data striping: Data is partitioned; size of a partition is called the striping unit. Partitions are distributed over several disks.
 - Redundancy: More disks -> more failures. Redundant information allows reconstruction of data if a disk fails.



RAID... (contd.)

- Data Striping Example...
 - D disks = D blocks are transferred simultaneously
 - Transfer Rate = D times faster
- An array of disks reduces reliability
 - Example
 - 1 disk (MTTF) $\sim 50,000$ hours
 - 100 disks $50,000 \text{ hours} / 100 \sim 21 \text{ days}$

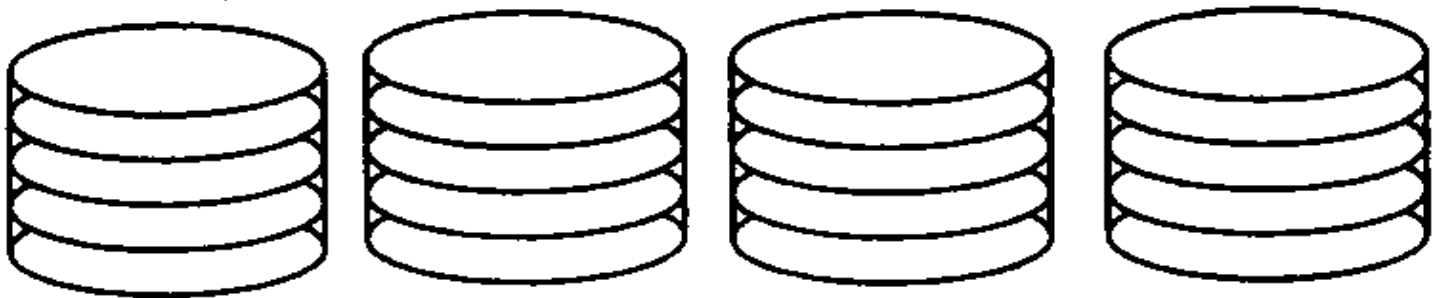


RAID... (contd.)

- In RAID, redundant information is stored to increase reliability
- Example.. parity scheme

RAID Levels

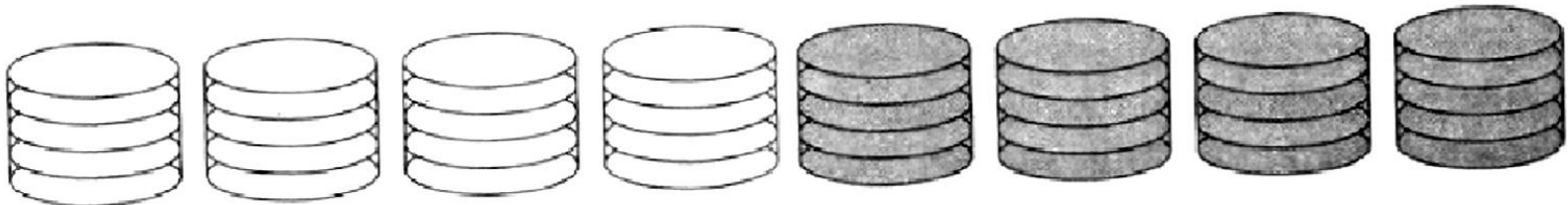
- Level 0: No redundancy
 - Uses data striping
 - Reliability an issue
 - Best Write Performance (No writing of redundant data)



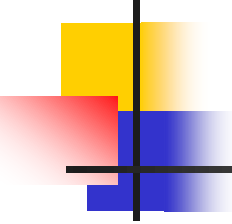
Non-Redundant (RAID Level 0)

RAID Levels... (contd.)

- Level 1: Mirrored (two identical copies)
 - No data striping
 - Most expensive solution
 - Each disk has a mirror image (check disk)
 - Parallel reads, a write involves two disks.
 - Maximum transfer rate = transfer rate of one disk (this level does not stripe the data).
 - Space utilization = 50%



Mirrored (RAID Level 1)

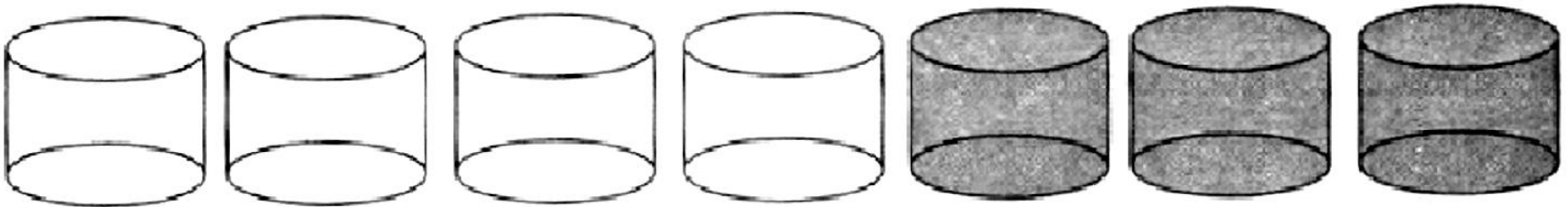


RAID Levels... (contd.)

- Level 0+1: Striping and Mirroring
 - Parallel reads, a write involves two disks.
 - Maximum transfer rate = aggregate bandwidth
 - Space utilization = 50%

RAID Levels... (contd.)

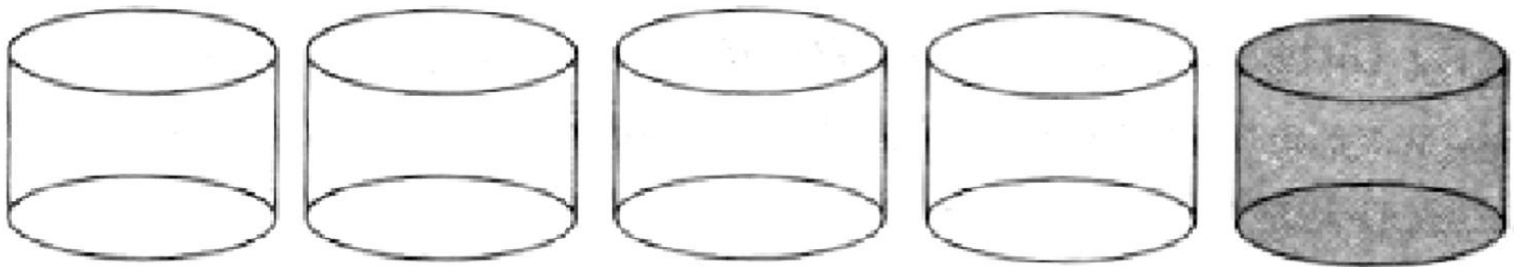
- Level 2: Error-Correcting Codes
 - Striping unit = 1 bit
 - Redundancy scheme = Hamming Code



Memory-Style ECC (RAID Level 2)

RAID Levels... (contd.)

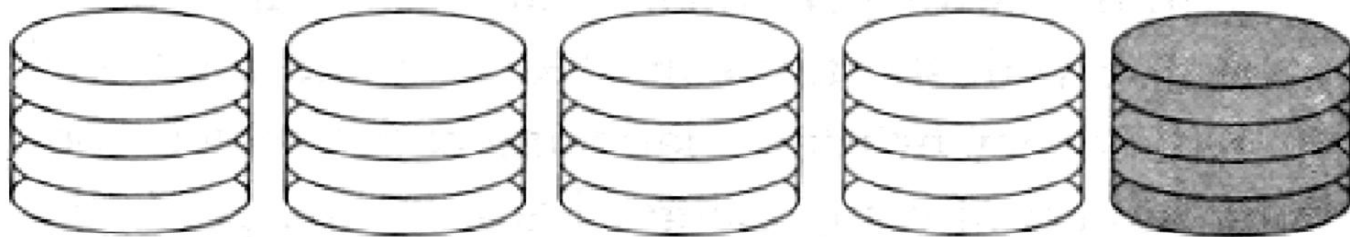
- Level 3: Bit-Interleaved Parity
 - Striping Unit: One bit. One check disk.
 - Each read and write request involves all disks; disk array can process one request at a time.



Bit-Interleaved Parity (RAID Level 3)

RAID Levels... (contd.)

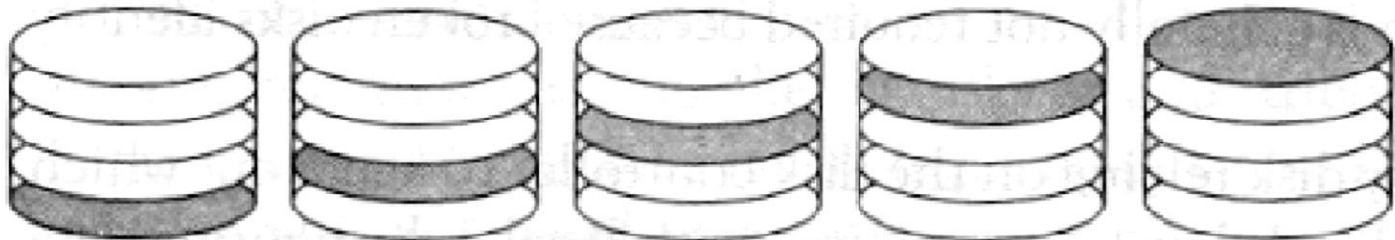
- Level 4: Block-Interleaved Parity
 - Striping Unit: One disk block. One check disk.
 - Parallel reads possible for small requests, large requests can utilize full bandwidth
 - Writes involve modified block and check disk



Block-Interleaved Parity (RAID Level 4)

RAID Levels... (contd.)

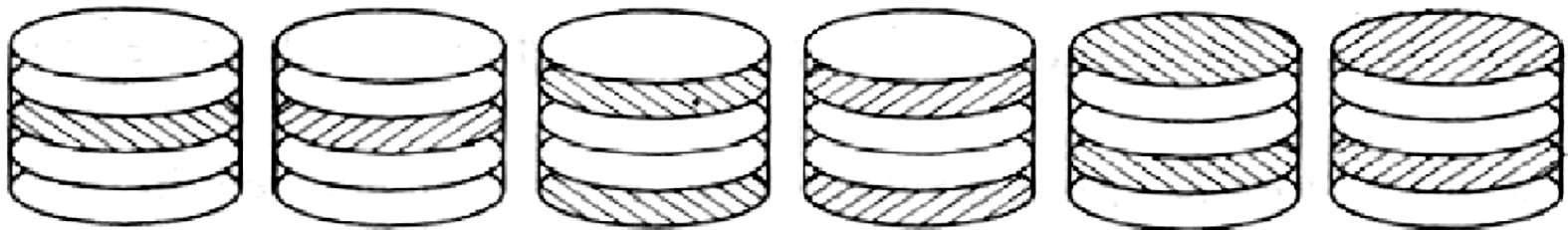
- Level 5: Block-Interleaved Distributed Parity
 - Similar to RAID Level 4, but parity blocks are distributed over all disks



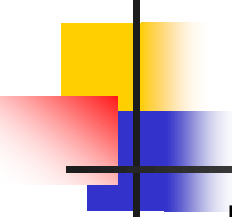
Block-Interleaved Distribution-Parity (RAID Level 5)

RAID Levels... (contd.)

- Level 6: P+Q Redundancy
 - Similar to RAID Level 5
 - Two check disks (uses Reed-Solomon codes)
 - Able to recover from 2 failures

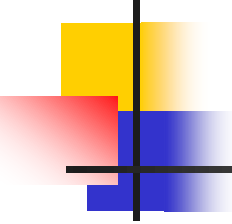


P+Q Redundancy (RAID Level 6)



Disk Space Management

- Lowest layer of DBMS software manages space on disk.
- Higher levels call upon this layer to:
 - allocate/de-allocate a page
 - read/write a page
- Page = 1 disk block
- 1 page request = 1 disk block
- Request for a *sequence* of pages must be satisfied by allocating the pages sequentially on disk!

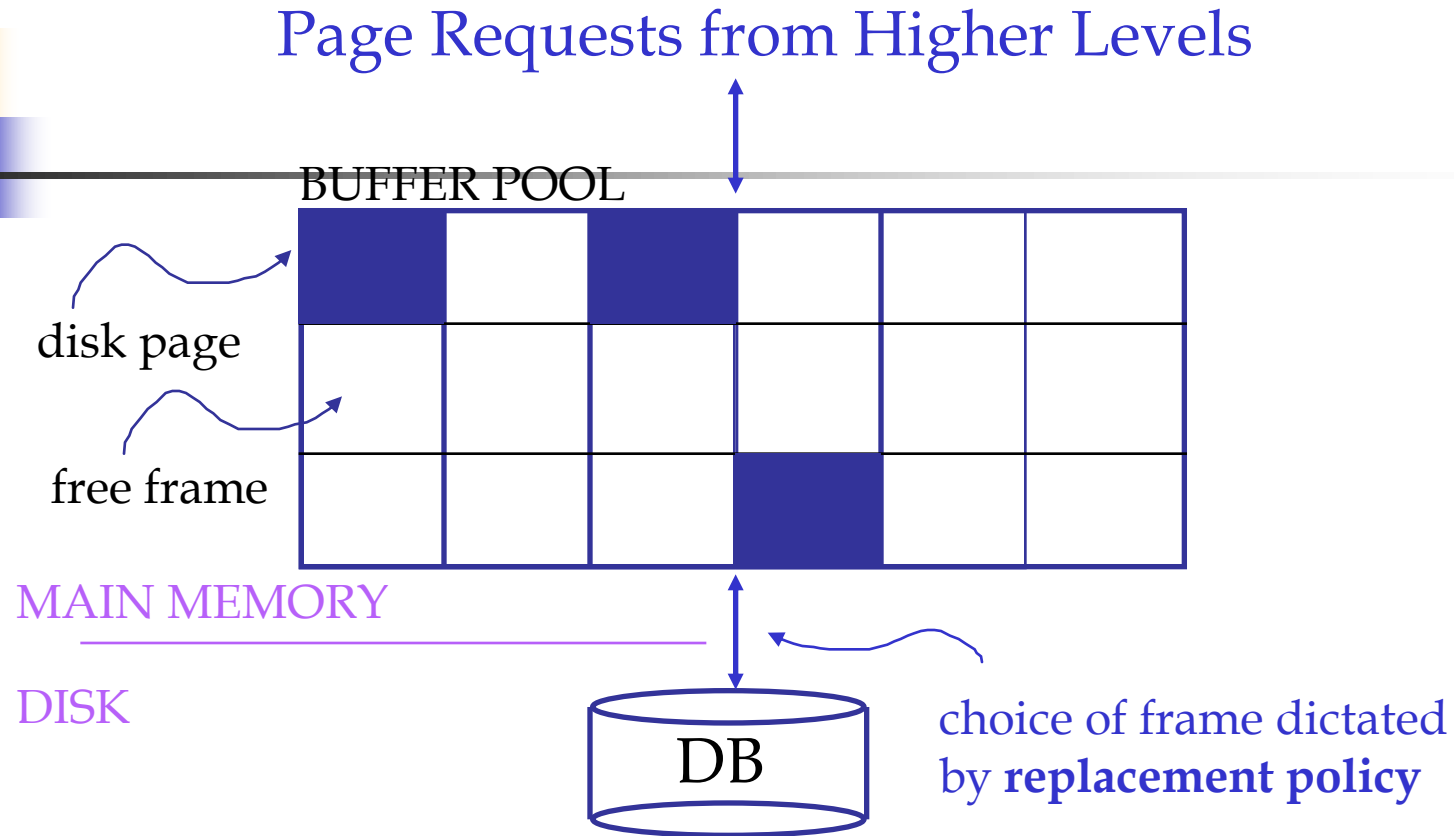


Disk Space Management...

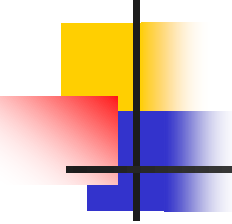
(contd.)

- Higher levels don't need to know how this is done, or how free space is managed.
- Maintaining free blocks...
 - Keeping a list of free blocks & a pointer to the first free block
 - Maintaining a Bitmap

Buffer Management in a DBMS

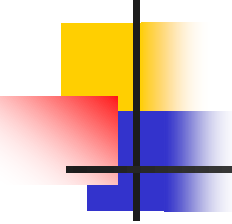


- *Data must be in RAM for DBMS to operate on it!*
- *Table of <frame#, pageid> pairs is maintained.*
- *For each frame#, 2 variables (pin count: no of current users & dirty bit: page is modified)*



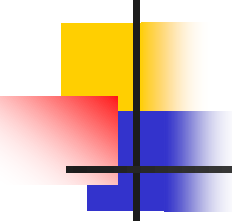
When a Page is Requested ...

- If exists then done
 - If requested page is not in pool:
 - Choose a frame for *replacement*
 - If frame is dirty, write it to disk
 - Read requested page into chosen frame
 - *Pin* the page and return its address.
- *If requests can be predicted (e.g., sequential scans) pages can be pre-fetched several pages at a time!*



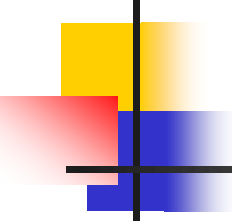
More on Buffer Management

- Requestor of page must unpin it, and indicate whether page has been modified:
 - *dirty* bit is used for this.
- Page in pool may be requested many times,
 - a *pin count* is used. A page is a candidate for replacement if and only if *pin count* = 0.
- CC & recovery may entail additional I/O when a frame is chosen for replacement. (*Write-Ahead Log* protocol; more later.)



Buffer Replacement Policy

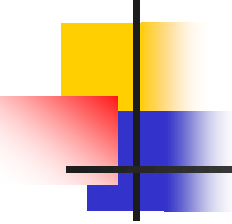
- Frame is chosen for replacement by a *replacement policy*:
 - Least-recently-used (LRU), Clock, MRU, random etc.
- Policy can have big impact on # of I/O's; depends on the *access pattern*.
- *Sequential flooding*: Nasty situation caused by LRU + repeated sequential scans.
 - # buffer frames < # pages in file means each page request causes an I/O. MRU much better in this situation (but not in all situations, of course).



DBMS vs. OS File System

OS does disk space & buffer mgmt: why not let OS manage these tasks?

- Differences in OS support: portability issues
- Some limitations, e.g., files can't span disks.
- Buffer management in DBMS requires ability to:
 - **pin a page** in buffer pool, **force a page** to disk (important for implementing CC & recovery),
 - adjust *replacement policy*, and **pre-fetch pages** based on access patterns in typical DB operations.



Summary

- Components of a DBMS
- Disks
- RAID
- Disk Space Management
- Buffer Management